

Lecture 19 - Runtime / ABI / Codegen

주제 : command buffer 와 tensor descriptor
ABI

목표 : compiler backend 와 edge runtime 의
계약을 명확히 정의

산출물 : NPU runtime ABI draft + command
buffer schema + lowering test

Lecture 19 Runtime / ABI / Codegen Pipeline



Compile-time decisions	tile shape, layout, quant, memory space, dependency graph
ABI decisions	tensor descriptor, command buffer, relocation table, versioning
Runtime decisions	device allocation, queue submit, fence, error handling, fallback

오늘의 위치

17 강 : fusion/layout 로 graph-level contract 확정

18 강 : custom NPU dialect 로 kernel/DMA/barrier 표현

19 강 : runtime 이 실행할 artifact 와 ABI 로 고정

20 강 : capstone design spec 로 통합

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Learning Objectives

Compiler/runtime responsibility boundary 설명
Tensor descriptor 와 command buffer field 설계
npu_rt pseudo dialect 와 runtime C ABI 연결
LLVM IR / EmitC / binary artifact codegen 선택지 비교
FileCheck 와 ABI manifest 기반 regression test 설계

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Compiler vs Runtime Responsibilities

Compiler: shape/layout/tile/fusion/quant/legal op 결정

Compiler: command dependency graph 와 descriptor metadata 생성

Runtime: device allocation, queue submit, fence, error handling

Runtime: capability query, fallback, tracing, profiling

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Final Lowering Boundary

npu_kernel.matmul/dma/barrier 는 아직 compiler-side semantic IR

npu_rt.* 는 runtime-visible ABI 를 만드는 마지막 dialect 이 boundary 이후에는 versioning, alignment, endianness, binary compatibility 가 중요

잘못 설계하면 compiler pass 하나가 runtime ABI 를 깨뜨림

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Runtime ABI = Contract

Function signature, tensor descriptor, command packet, error code 를 포함

Host CPU, firmware, driver, compiler 가 공유하는 안정된 문서
IR 내부 최적화 정보와 runtime-visible 정보를 분리

ABI version 과 target capability 를 항상 명시

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

ABI Design Rules

Fixed-width integer type 사용 : uint32_t/uint64_t
모든 struct 는 alignment 와 padding 을 명시
reserved field 를 남겨 forward compatibility 확보
pointer 대신 device address/handle/descriptor index 사용
debug tag 와 checksum 은 optional but useful

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Tensor Descriptor Overview

Tensor data 를 runtime 이 이해하는 최소 metadata
base address + offset + shape + stride + dtype + layout
quantization scale/zero point/axis, memory space, flags 포함 가능
compiler 와 runtime 이 같은 descriptor schema 를 공유해야 함

Tensor Descriptor ABI Layout

magic/version	0x4E505544 / ABI v1
rank/dtype/layout	rank=4, dtype=i8, layout=NHWC
shape[0..7]	N,H,W,C plus padding
stride[0..7]	physical addressing stride
base/offset	device address + byte offset
memory_space	DRAM/SRAM/ACCUM/CONST
quant	scale, zero point, axis
flags	readonly, packed, host visible

NPU rule: descriptor is runtime-visible ABI; compiler must not leak unstable IR-only assumptions into this format.

Tensor Descriptor Fields

magic/version: descriptor type 확인

rank/shape/stride: physical addressing

dtype/layout/memory_space: hardware interpretation

quant: per-tensor/per-channel metadata

flags: readonly, packed, host-visible, constant weight

NPU 관점

- address generation
- layout/stride check
- quant metadata
- runtime ABI field

Layout and Stride Contract

Logical shape 와 physical stride 를 분리

NHWC/NCHW 같은 label 만으로는 DMA address generation 이 부족

stride 는 byte 또는 element 단위 중 하나로 고정

packed INT4/INT2 는 element stride 와 byte packing rule 을 별도 표기

NPU 관점

- address generation
- layout/stride check
- quant metadata
- runtime ABI field

Memory Space Contract

DRAM: external tensor/weight/activation buffer

SRAM: tile staging buffer, runtime 이 보통 직접 노출하지 않음

ACCUM: compute array accumulator, command-local object
일 수 있음

CONST: immutable weight/lookup table memory

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Quant Metadata Contract

scale, zero_point, quant_axis, storage_type,
expressed_type

per-channel 이면 scale pointer 또는 descriptor table index
필요

requant multiplier/shift 는 command field 로 돌지
descriptor field 로 돌지 결정

runtime 이 numeric policy 를 재해석하지 않도록 compiler 가
명시

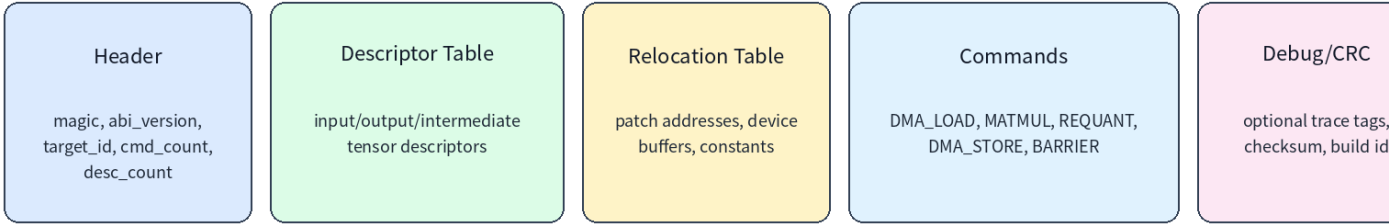
NPU 관점

- address generation
- layout/stride check
- quant metadata
- runtime ABI field

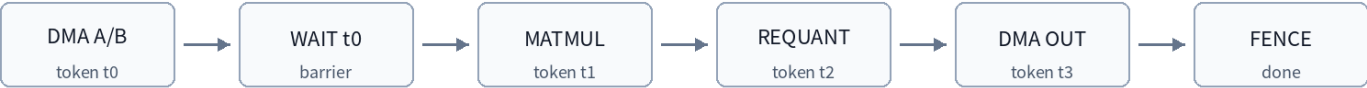
Command Buffer Overview

NPU 가 실행할 command packet sequence
Header + descriptor table + relocation table +
commands + optional debug block
offline compiled artifact 또는 runtime-
generated artifact 로 사용
queue submit API 가 command buffer handle
을 받음

Command Buffer Format



Execution order and dependencies



Command Buffer Header

magic, abi_version, target_id

command_count, descriptor_count, relocation_count

byte_size, alignment, checksum

feature flags: profiling, debug, priority, secure mode

Packet 관점

- descriptor index
- token dependency
- alignment
- debug trace

Command Types

DMA_LOAD / DMA_STORE: memory movement

MATMUL / CONV / ELEMENTWISE: compute packet

REQUANT / EPILOGUE: numeric post-processing

BARRIER / WAIT / SIGNAL: dependency and synchronization

NOP / TRACE: debug and alignment padding

Packet 관점

- descriptor index
- token dependency
- alignment
- debug trace

Dependency / Token Model

Sequential command stream 만으로는 overlap 표현이 어렵다
Token 또는 fence field 로 DMA/compute dependency 를
명시

runtime scheduler 는 token graph 를 보고 overlap 가능
검증기는 dead token, unsatisfied dependency, cycle 을
찾아야 함

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

DMA Command Packet

src descriptor index, dst descriptor index
src/dst offset, byte count, stride/pitch
direction: host<->device, DRAM<->SRAM, SRAM<->SRAM
completion token 과 barrier dependency

Packet 관점

- descriptor index
- token dependency
- alignment
- debug trace

Compute Command Packet

op kind: matmul/conv/attention/custom
operand descriptor indices
tile shape, layout, accumulator type
epilogue flags: bias, relu, gelu, requant
microcode/kernel id 또는 hardware opcode

Packet 관점

- descriptor index
- token dependency
- alignment
- debug trace

Barrier and Fence Packet

barrier 는 command graph 내부 dependency

fence 는 host/runtime visibility boundary

firmware queue 와 host queue 가 다른 경우 fence granularity 가 중요

timeout/error code/profiling timestamp 와 연결

Packet 관점

- descriptor index
- token dependency
- alignment
- debug trace

Descriptor Table and Relocation

Descriptor table: command 에서 index 로 참조

Relocation table: device allocation 이후 patch 할 address 위치

Weight constant 는 artifact 에 포함하거나 external handle 로 참조

AOT artifact 는 relocation 을 분리해야 재사용 가능

NPU 관점

- address generation
- layout/stride check
- quant metadata
- runtime ABI field

Compilation Artifact Contents

model.json or manifest.yaml
command_buffer.bin
tensor_descriptors.bin
constants.bin
debug_symbols.json / profiling map
ABI version, compiler build id, target feature list

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Runtime API Surface

npuInit / npuCreateContext / npuDestroyContext
npuLoadModule / npuGetTensorDesc / npuBindTensor
npuSubmit / npuWait / npuGetStatus
npuQueryCapability / npuGetProfilingTrace

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

C ABI Header Principle

extern "C" API 로 C++ ABI 문제를 피함

opaque handle: NpuContext, NpuModule, NpuQueue

caller-owned vs runtime-owned memory ownership 명시

return value 는 status enum + optional diagnostic string

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

npu_rt Pseudo Dialect

npu_rt.module: target/ABI artifact scope

npu_rt.tensor_desc: runtime-visible tensor metadata

npu_rt.command_buffer: command sequence region

npu_rt.call_runtime: runtime API call boundary

npu_rt.emit_artifact: binary/manifest emission point

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Lowering npu_rt to Runtime Calls

npu_rt.tensor_desc -> constant struct or descriptor table entry

npu_rt.command_buffer -> binary blob or C initializer

npu_rt.submit -> npuSubmit(ctx, queue, cb, bindings)

npu_rt.wait -> npuWait(fence, timeout_ns)

errors become status code and diagnostics

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Codegen Choices

LLVM dialect -> LLVM IR: host stub/JIT/prototype 에 좋음

EmitC -> C/C++: embedded static build 에 적합

Binary command buffer: runtime submit path 가 단순

Hybrid: C stub + binary command buffer + manifest 가 가장 실용적

Codegen Choices for Edge NPU Runtime

LLVM Dialect -> LLVM IR

native host stubs, JIT/prototype, C-compatible wrappers, memref descriptor lowering

EmitC -> C/C++

embedded-friendly generated C, static build, easy cross compile

Binary Command Buffer

no host codegen needed, compile-time artifact submitted to runtime

Hybrid

C runtime stub + binary command buffer + metadata manifest

MLIR to LLVM Flow

- 1) MLIR dialect 를 LLVM dialect 로 progressive conversion
- 2) llvm.func/llvm.mlir.global 등 translatable op 로 정리
- 3) mlir-translate 로 LLVM IR 생성
- 4) clang/lld/linker 로 host runtime stub 빌드

Codegen 관점

- LLVM host stub
- EmitC static C
- binary artifact
- hybrid deployment

EmitC Flow

npu_rt call 을 emitc.call_opaque 로 낮춤
descriptor 와 command buffer 를 C array initializer 로 표현
mlir-translate -mlir-to-cpp 로 C/C++ 출력
embedded cross compile 과 runtime integration 이 쉬움

Codegen 관점

- LLVM host stub
- EmitC static C
- binary artifact
- hybrid deployment

Embedded / Edge Constraints

No JIT, limited dynamic allocation, deterministic latency
artifact size 와 alignment 중요

firmware/runtime update 와 compiler version mismatch
대비

secure boot / signed model artifact / CRC 검증 고려

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Error Handling and Versioning

ABI mismatch, unsupported feature, descriptor invalid,
command invalid
runtime status code 와 compiler diagnostic 을 분리
major.minor ABI version 사용
reserved bits 와 extension blocks 로 forward compatibility
확보

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Capability Query and Fallback

runtime 이 target feature 를 제공 : tile sizes, dtype, memory, firmware version

compiler 는 target profile 과 runtime capability 를 matching

unsupported op 는 CPU/GPU fallback 또는 compile failure
fallback boundary 는 tensor descriptor ABI 와 일치해야 함

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Testing ABI with FileCheck

npu_rt op 가 descriptor/command field 를 정확히 생성하는지
검사

negative test: unsupported dtype/layout/unaligned
descriptor

binary blob 은 manifest JSON/YAML 로 deterministic 하게
검증

golden runtime simulator 로 command sequence sanity
check

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Debugging Runtime/Codegen

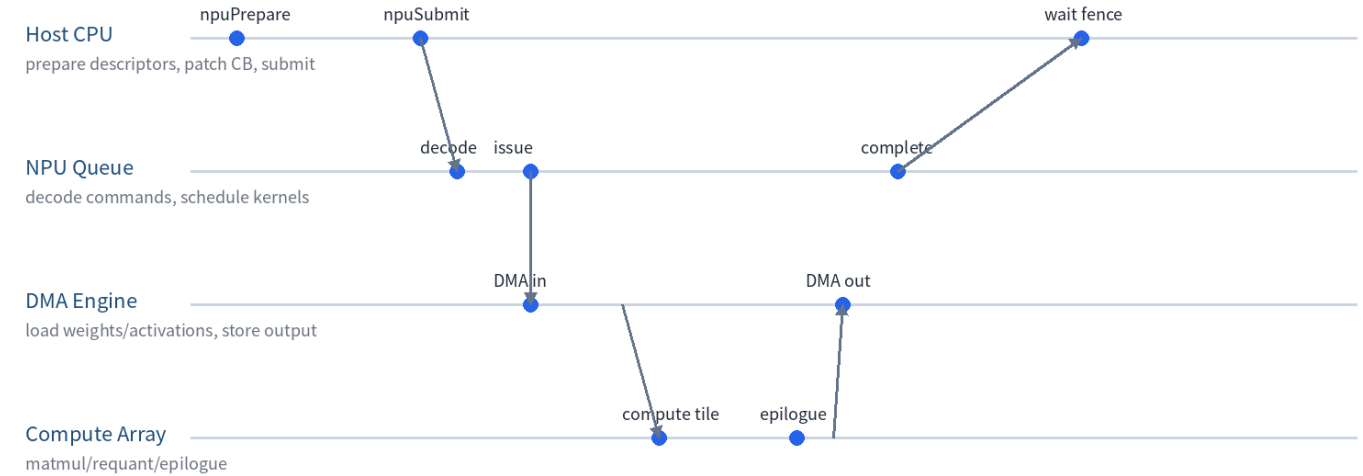
descriptor dump: shape/stride/memory
space/quant 확인

command trace: issue time, done time, token
dependency

firmware log 와 compiler debug symbol map
연결

minimum reproducible command buffer 를
저장

Runtime Execution Timeline



Case Study: MatMul Tile CB

Input A/B descriptor + output descriptor + temporary SRAM descriptors

DMA_LOAD A/B -> WAIT -> MATMUL -> REQUANT ->
DMA_STORE -> FENCE

tile M/N/K 와 accumulator type 이 compute command 에
들어감

descriptor index 와 token id 가 command packet 의 핵심

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Exercise A - Tensor Descriptor

MatMul input/output descriptor field 를 직접 채우기
shape, stride, dtype, layout, memory_space, quant field
결정
descriptor alignment 와 byte_size 계산
runtime-visible field 와 compiler-only field 분리

NPU 관점

- address generation
- layout/stride check
- quant metadata
- runtime ABI field

Exercise B - Command Buffer

DMA A/B, MATMUL, REQUANT, DMA OUT, FENCE command packet 작성

각 command 의 dependency token 표시
descriptor index 와 relocation entry 연결
minimum validation checklist 작성

Packet 관점

- descriptor index
- token dependency
- alignment
- debug trace

Exercise C - Lowering Path

npu_rt.command_buffer 를 LLVM/EmitC/binary 중 하나로 lowering

C API wrapper signature 정의

FileCheck positive/negative test 작성

fallback policy 와 capability query 를 manifest 에 반영

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

Takeaways

Runtime ABI 는 마지막 IR 보다 더 오래 살아남는 계약

Tensor descriptor 와 command buffer 는 NPU product 품질을 좌우

MLIR codegen 은 LLVM IR, EmitC, binary artifact 등 여러 선택지가 가능

19 강 산출물이 20 강 capstone 의 runtime/codegen 섹션이 됨

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?

References

MLIR LLVM IR Target: two-stage lowering to LLVM dialect then LLVM IR

MLIR LLVM Dialect: MLIR mapping of LLVM IR operations/types

MLIR EmitC Dialect: MLIR-to-C/C++ generation path

MLIR C API: C-level interop and opaque handle design

핵심 질문

- 무엇이 ABI 인가 ?
- 누가 결정하는가 ?
- runtime 이 재해석하는가 ?
- 어떻게 테스트하는가 ?